

Q1) Priority Encoder

```
1  module priority_enc_tb();
2
3  logic  clk;
4  logic  rst;
5  logic  [3:0] D;
6  logic  [1:0] Y_dut;
7  logic  valid_dut;
8
9
10 priority_enc dut (.clk(clk), .rst(rst), .D(D), .Y(Y_dut), .valid(valid_dut));
11 bind priority_enc assertions_tb assertions_inst(
12     .clk(clk),
13     .rst(rst),
14     .D(D),
15     .Y(Y),
16     .valid(valid)
17 );
18
19
20 initial begin
21     clk = 0;
22     forever
23         #1 clk = ~clk;
24 end
25
26
27 task reset_assert();
28     rst = 1;
29     @(negedge clk);
30     assert (Y_dut == 2'b00 && valid_dut == 1'b0) else $error("Reset Failed");
31     rst = 0;
32 endtask
33
34 initial begin
35
36
37     //Priority_Encoder_1
38     reset_assert();
39
40     //Priority_Encoder_2
41     D = 4'b0000; @(negedge clk); //Y_expected is Don't Care
42     D = 4'b1000; @(negedge clk);
43
44     D = 4'b0100; @(negedge clk);
45     D = 4'b1100; @(negedge clk);
46
47     D = 4'b0010; @(negedge clk);
48     D = 4'b0110; @(negedge clk);
49     D = 4'b1010; @(negedge clk);
50     D = 4'b1110; @(negedge clk);
51
52     D = 4'b0001; @(negedge clk);
53     D = 4'b0011; @(negedge clk);
54     D = 4'b0101; @(negedge clk);
55     D = 4'b0111; @(negedge clk);
56     D = 4'b1001; @(negedge clk);
57     D = 4'b1011; @(negedge clk);
58     D = 4'b1101; @(negedge clk);
59     D = 4'b1111; @(negedge clk);
60
61     D = 4'b1000; @(negedge clk); //For 100% Code Coverage
62
63     //Priority_Encoder_1
64     reset_assert();
65
66     $stop;
67
68 end
69
70 endmodule
```

```

71
72 module assertions_tb(input logic clk, input logic rst, input logic [3:0] D, input logic [1:0] Y, input logic valid);
73
74     property p_D_bit_3;
75     @(posedge clk) disable iff (rst) (D[3] && !D[2] && !D[1] && !D[0]) |> (Y == 2'b00 && valid == 1'b1);
76     endproperty
77
78     property p_D_bit_2;
79     @(posedge clk) disable iff (rst) (!D[3] && D[2] && !D[1] && !D[0]) |> (Y == 2'b01 && valid == 1'b1);
80     endproperty
81
82     property p_D_bit_1;
83     @(posedge clk) disable iff (rst) (!D[3] && !D[2] && D[1] && !D[0]) |> (Y == 2'b10 && valid == 1'b1);
84     endproperty
85
86     property p_D_bit_0;
87     @(posedge clk) disable iff (rst) (!D[3] && !D[2] && !D[1] && D[0]) |> (Y == 2'b11 && valid == 1'b1);
88     endproperty
89
90     property p_invalid;
91     @(posedge clk) disable iff (rst) (D == 4'b0000) |> (valid == 1'b0);
92     endproperty
93
94
95
96     D_bit_3_Assertion: assert property (p_D_bit_3) else $error("D[3] Assertion Failed");
97     D_bit_2_Assertion: assert property (p_D_bit_2) else $error("D[2] Assertion Failed");
98     D_bit_1_Assertion: assert property (p_D_bit_1) else $error("D[1] Assertion Failed");
99     D_bit_0_Assertion: assert property (p_D_bit_0) else $error("D[0] Assertion Failed");
100    Invalid_Assertion: assert property (p_invalid) else $error("Invalid Assertion Failed");
101
102    D_bit_3_Coverage: cover property (p_D_bit_3);
103    D_bit_2_Coverage: cover property (p_D_bit_2);
104    D_bit_1_Coverage: cover property (p_D_bit_1);
105    D_bit_0_Coverage: cover property (p_D_bit_0);
106    Invalid_Coverage: cover property (p_invalid);
107 endmodule

```

Assertions Tab

Name	Assertion Type	Language	Enable	Failure Count	Pass Count	A
/priority_enc_tb/dut/assertions_inst/D_bit_3_Assertion	Concurrent	SVA	on	0	1	
/priority_enc_tb/dut/assertions_inst/D_bit_2_Assertion	Concurrent	SVA	on	0	1	
/priority_enc_tb/dut/assertions_inst/D_bit_1_Assertion	Concurrent	SVA	on	0	1	
/priority_enc_tb/dut/assertions_inst/D_bit_0_Assertion	Concurrent	SVA	on	0	1	
/priority_enc_tb/dut/assertions_inst/Invalid_Assertion	Concurrent	SVA	on	0	1	
/priority_enc_tb/reset_assert/immed__30	Immediate	SVA	on	0	1	

Assertions Coverage

ame	Language	Enabled	Log	Count	AtLeast	Limit	Weight	Cmplt %	Cmplt graph
/priority_enc_tb/dut/assertions_inst/D_bit_3_Coverage	SVA	✓	Off	1	1	Unli...	1	100%	
/priority_enc_tb/dut/assertions_inst/D_bit_2_Coverage	SVA	✓	Off	1	1	Unli...	1	100%	
/priority_enc_tb/dut/assertions_inst/D_bit_1_Coverage	SVA	✓	Off	1	1	Unli...	1	100%	
/priority_enc_tb/dut/assertions_inst/D_bit_0_Coverage	SVA	✓	Off	1	1	Unli...	1	100%	
/priority_enc_tb/dut/assertions_inst/Invalid_Coverage	SVA	✓	Off	1	1	Unli...	1	100%	

ALU:

Test bench:

```
1 module ALU_4_bit_tb();
2
3 logic clk;
4 logic reset;
5 logic [1:0] Opcode; // The opcode
6 logic signed [3:0] A; // Input data A in 2's complement
7 logic signed [3:0] B; // Input data B in 2's complement
8
9 logic signed [4:0] C_dut; // ALU output in 2's complement
10
11 localparam MAXPOS = 7;
12 localparam MAXNEG = -8;
13
14 localparam Add = 2'b00; // A + B
15 localparam Sub = 2'b01; // A - B
16 localparam Not_A = 2'b10; // ~A
17 localparam ReductionOR_B = 2'b11; // |B
18
19 int correct_count, error_count;
20
21 ALU_4_bit dut(clk, reset, Opcode, A, B, C_dut);
22
23 bind ALU_4_bit assertions_tb assertions_tb_inst(.clk(clk), .reset(reset), .Opcode(Opcode), .A(A), .B(B), .C(C));
24
25 initial begin
26     clk = 0;
27     forever
28         #1 clk = ~clk;
29 end
30
31 task reset_assert();
32     reset = 1;
33     @(negedge clk);
34     reset = ~reset;
35 endtask
36
37
38 initial begin
39     A = 0;
40     B = 0;
41     Opcode = 0;
42
43     correct_count = 0;
44     error_count = 0;
45
46     //ALU_1
47     reset_assert();
48     //ALU_2
49     A = MAXPOS; B = MAXPOS; Opcode = Add; @(negedge clk);
50     //ALU_3
51     A = MAXPOS; B = MAXNEG; Opcode = Add; @(negedge clk);
52     //ALU_4
53     A = MAXNEG; B = MAXPOS; Opcode = Add; @(negedge clk);
54     //ALU_5
55     A = MAXNEG; B = MAXNEG; Opcode = Add; @(negedge clk);
56     //ALU_6
57     A = MAXPOS; B = MAXPOS; Opcode = Sub; @(negedge clk);
58     //ALU_7
59     A = MAXPOS; B = MAXNEG; Opcode = Sub; @(negedge clk);
60     //ALU_8
61     A = MAXNEG; B = MAXPOS; Opcode = Sub; @(negedge clk);
62     //ALU_9
63     A = MAXNEG; B = MAXNEG; Opcode = Sub; @(negedge clk);
64     //ALU_10
65     A = 4'b1111; Opcode = Not_A; @(negedge clk);
66     //ALU_11
67     B = 4'b0000; Opcode = ReductionOR_B; @(negedge clk);
68     B = 4'b0001; Opcode = ReductionOR_B; @(negedge clk);
69     B = 4'b0010; Opcode = ReductionOR_B; @(negedge clk);
70     B = 4'b0100; Opcode = ReductionOR_B; @(negedge clk);
71     B = 4'b1000; Opcode = ReductionOR_B; @(negedge clk);
72     B = 4'b1111; Opcode = ReductionOR_B; @(negedge clk);
73
74
75     A = 4; B = -6; Opcode = Add; @(negedge clk);
76
77     $display("Total Tests:%d | Correct Tests:%d | Error Test:%d", correct_count + error_count, correct_count, error_count);
78     $stop;
79
80 end
81
82 endmodule
83
```

```

84
85 module assertions_tb (
86     input logic clk,
87     input logic reset,
88     input logic [1:0] Opcode,
89     input logic signed [3:0] A,
90     input logic signed [3:0] B,
91     input logic signed [4:0] C
92 );
93
94 // Property for Addition (Opcode = 00)
95 property p_add;
96     @(posedge clk) disable iff (reset)
97         ($past(Opcode) == 2'b00) |-> (C == ($past(A) + $past(B)));
98 endproperty
99
100 // Property for Subtraction (Opcode = 01)
101 property p_sub;
102     @(posedge clk) disable iff (reset)
103         ($past(Opcode) == 2'b01) |-> (C == ($past(A) - $past(B)));
104 endproperty
105
106 // Property for Bitwise NOT of A (Opcode = 10)
107 property p_not_a;
108     @(posedge clk) disable iff (reset)
109         ($past(Opcode) == 2'b10) |-> (C == (~$past(A)));
110 endproperty
111
112 // Property for Reduction OR of B (Opcode = 11)
113 property p_red_or_b;
114     @(posedge clk) disable iff (reset)
115         ($past(Opcode) == 2'b11) |-> (C == (!$past(B)));
116 endproperty
117
118 // Property for Reset behavior
119 property p_reset;
120     @(posedge clk)
121         reset |-> (C == 5'b0);
122 endproperty
123
124 always_comb begin
125     if (reset) begin
126         assert final (C == 5'b0) else $error("Reset Assertion Failed: C is not zero on reset");
127         cover final (C == 5'b0);
128     end
129 end
130
131
132 // Assertions
133 Add_Assertion:    assert property (p_add)           else $error("Addition Assertion Failed");
134 Sub_Assertion:    assert property (p_sub)           else $error("Subtraction Assertion Failed");
135 Not_A_Assertion:  assert property (p_not_a)         else $error("NOT A Assertion Failed");
136 Red_OR_B_Assertion: assert property (p_red_or_b)    else $error("Reduction OR of B Assertion Failed");
137 Reset_Assertion:  assert property (p_reset)         else $error("Reset Assertion Failed");
138
139 // Coverage
140 Add_Coverage:     cover property (p_add);
141 Sub_Coverage:     cover property (p_sub);
142 Not_A_Coverage:   cover property (p_not_a);
143 Red_OR_B_Coverage: cover property (p_red_or_b);
144 Reset_Coverage:   cover property (p_reset);
145
146 endmodule
147
148

```

Assertions Tab:

Name	Assertion Type	Language	Enable	Failure Count	Pass Count	Active
 /ALU_4_bit_tb/dut/assertions_tb_inst/Add_Assertion	Concurrent	SVA	on	0	1	
 /ALU_4_bit_tb/dut/assertions_tb_inst/Sub_Assertion	Concurrent	SVA	on	0	1	
 /ALU_4_bit_tb/dut/assertions_tb_inst/Not_A_Assertion	Concurrent	SVA	on	0	1	
 /ALU_4_bit_tb/dut/assertions_tb_inst/Red_OR_B_Assertion	Concurrent	SVA	on	0	1	
 /ALU_4_bit_tb/dut/assertions_tb_inst/Reset_Assertion	Concurrent	SVA	on	0	1	
 /ALU_4_bit_tb/dut/assertions_tb_inst/#ublk#151805794#125/immed__126	Immediate	SVA	on	0	1	

Assertion Coverage

	Language	Enabled	Log	Count	AtLeast	Limit	Weight	Cmplt %	Cmplt graph
/ALU_4_bit_tb/dut/assertions_tb_inst/#ublk#151805794#125/i_cover__127	SVA	✓	Off	1	1	Unli...	1	100%	
/ALU_4_bit_tb/dut/assertions_tb_inst/Add_Coverage	SVA	✓	Off	5	1	Unli...	1	100%	
/ALU_4_bit_tb/dut/assertions_tb_inst/Sub_Coverage	SVA	✓	Off	4	1	Unli...	1	100%	
/ALU_4_bit_tb/dut/assertions_tb_inst/Not_A_Coverage	SVA	✓	Off	1	1	Unli...	1	100%	
/ALU_4_bit_tb/dut/assertions_tb_inst/Red_OR_B_Coverage	SVA	✓	Off	6	1	Unli...	1	100%	
/ALU_4_bit_tb/dut/assertions_tb_inst/Reset_Coverage	SVA	✓	Off	1	1	Unli...	1	100%	

Q2)

```

Digital_Verification_Diploma > Session_4 > Extra_Assignment > Q2 > arbiter_SVA.sv > arbiter_SVA
1  module arbiter_SVA();
2
3      property p_request_grant;
4          @(posedge clk) $rose(request) |-> ##[2:5] $rose(grant);
5      endproperty
6
7      property p_grant_ack;
8          @(posedge clk) $rose(grant) |-> ($fell(frame) && $fell(irdy));
9      endproperty
10
11     property p_complete_transaction;
12         @(posedge clk) ($rose(frame) && $rose(irdy)) |=> $fell(grant);
13     endproperty
14
15 endmodule

```

Q3)

```

1  module FSM_SVA();
2
3      property p_current_onehot;
4          @(posedge clk) $onehot(cs);
5      endproperty
6
7      //Assuming that "IDLE", "GEN_BLK_ADDR" and "WAIT0" are predefined parameters
8      property p_idle_get_data;
9          @(posedge clk) ((cs == IDLE) && $rose(get_data)) |-> ##1 (cs == GEN_BLK_ADDR) ##64 (cs == WAIT0);
10     endproperty
11
12 endmodule

```